

Fuse模块学习

FUSE (Filesystem in Userspace) 是一个允许用户在用户态创建自定义文件系统的接口，诞生于 2001 年。FUSE 的出现大大降低了文件系统开发的门槛，使得开发者能够在不修改内核代码的情况下实现创新的文件系统功能。JuiceFS 就是基于 FUSE 构建的高性能分布式文件系统，充分发挥了 FUSE 的灵活性和扩展性。

为了更好地理解 FUSE 的设计理念，我们将首先回顾内核文件系统以及网络文件系统（如 NFS）的发展历程，这些技术的演进为 FUSE 实现用户空间文件系统功能奠定了重要基础。

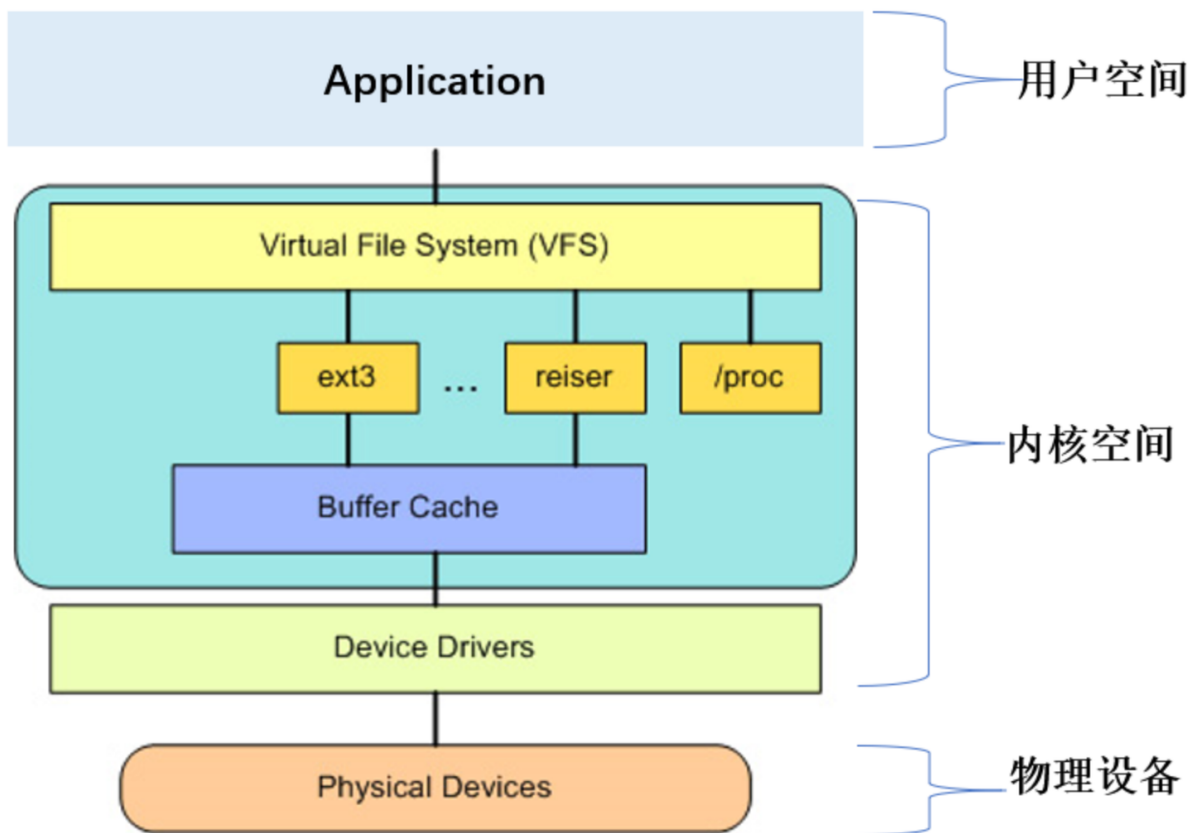
在文章的最后，将介绍 JuiceFS 在使用 FUSE 过程中的实践。由于 FUSE 需要进行用户态与内核态之间的切换，这会带来一定的开销，并可能导致 I/O 延迟，因此许多人对其性能存在顾虑。从我们的实践经验来看，FUSE 在大多数 AI 场景下能够满足性能需求，相关细节将在文章中阐述。

01 单机文件系统：内核态与 [VFS](#)

文件系统作为操作系统中的核心底层组件，负责频繁操作存储设备，因此最初的设计完全在内核空间中进行。“内核”这一概念的提出，是随着计算机硬件和软件日益复杂化，操作系统将底层资源管理的代码与用户程序进行分离的产物。

内核是拥有超级权限的代码，负责管理计算机的核心资源（如 CPU、内存、硬盘、网络等）。当内核代码运行时，程序进入内核态，可以完全访问和操作这些底层设备。由于内核态权限极高，其代码必须经过严格测试和验证，普通用户无法随意修改。

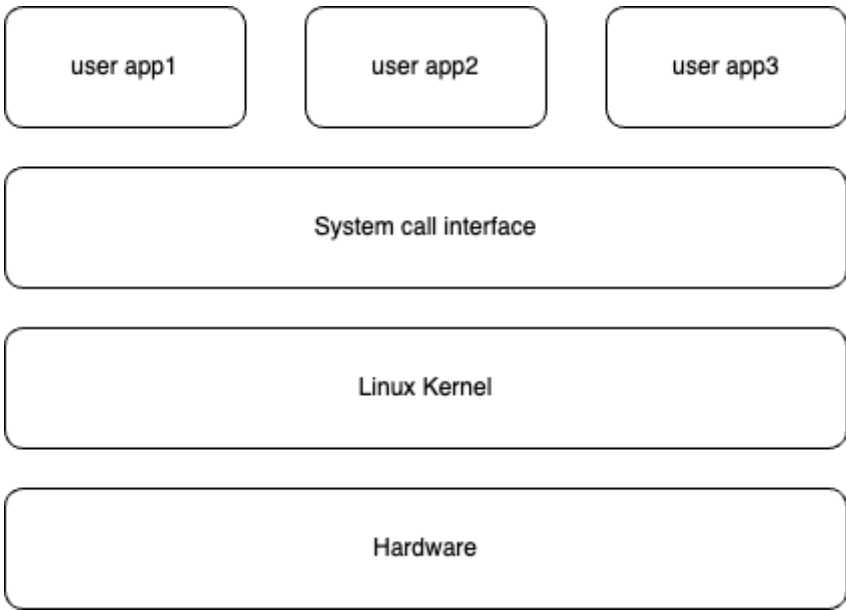
而与内核空间形成对应的是用户空间（User space），这部分代码就是我们平时常见的各种应用程序，像浏览器、游戏等。**在用户空间里，程序的权限是受到严格限制的，不能直接访问底层的重要资源。**



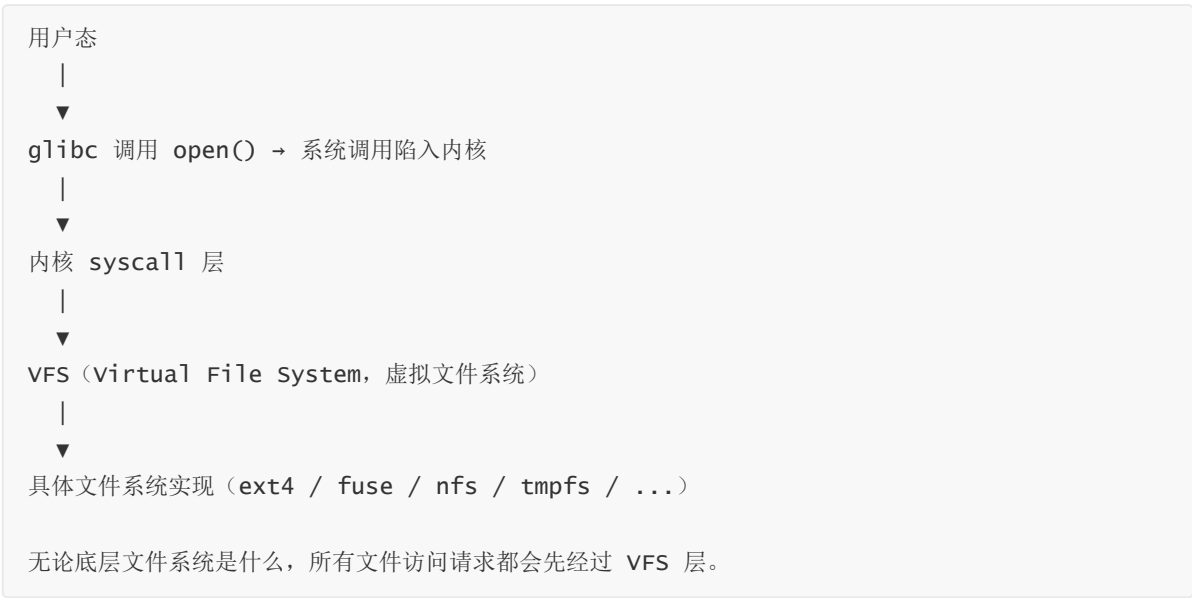
【VFS在内核空间】

如果应用程序需要使用文件系统，必须通过操作系统设计好的接口来进行访问，比如我们常用的 OPEN、READ、WRITE 等，这些就是所谓的系统调用。**系统调用的作用是搭建起用户空间和内核空间的桥梁。**现代操作系统常常定义了数百个系统调用，每个调用都有自己明确的名称、编号和参数。

当应用程序调用系统调用时，它就会进入一段内核空间代码，等执行完毕后，再把结果返回给用户空间。值得注意的是，这个从用户态到内核态，然后再返回用户态的整个过程，它都属于同一个进程范畴。



例子



在了解上述背景知识后，接下来我们将简要说明当用户调用文件系统接口时，用户态与内核态是如何实现交互的。

内核通过 VFS (virtual file system) 封装了一套通用的虚拟文件系统接口，向上以系统调用的形式暴露给用户态，向下给底层的文件系统规定编程接口，底层文件系统需要按照 VFS 的格式来实现自己的文件系统接口。因此，用户态访问底层文件系统的标准流程一般是 **【系统调用-> VFS -> 底层文件系统 -> 物理设备】**。

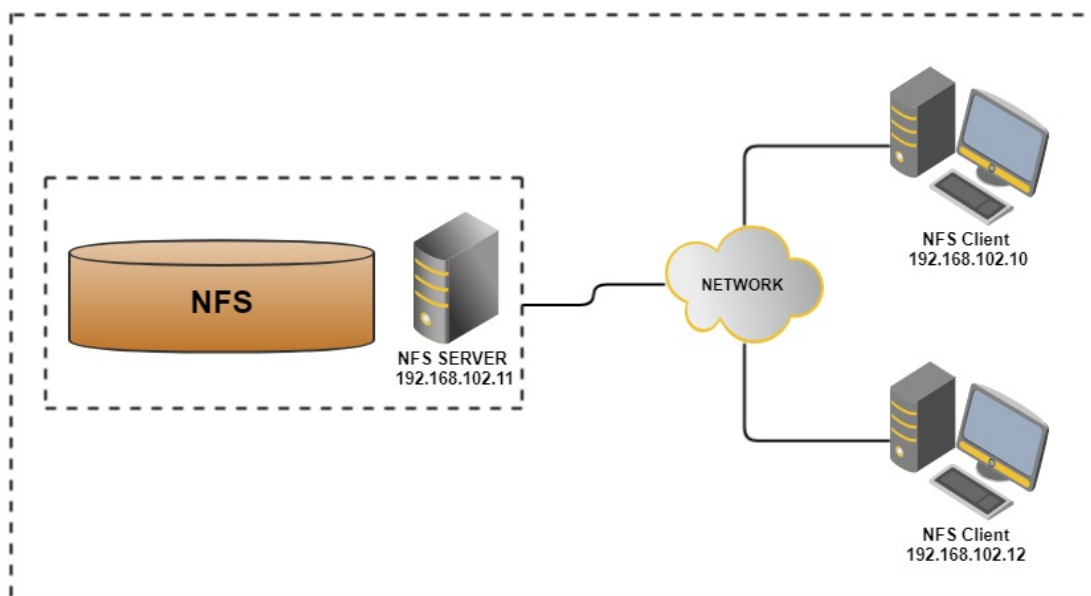
例如，我们在应用程序中调用 open，它会携带一个路径作为它的参数。这个调用到达 VFS 层以后，VFS 会根据这个路径，在它的树状结构中逐级查找，最终找到一个对应的目标和它所属的底层文件系统，并且这个底层文件系统也有一个自己实现的 open 方法，然后把这个 open 调用传递给底层文件系统。

Linux 内核支持数十种不同的文件系统。对于内存或网络等不同的存储介质，会采用不同的文件系统来进行管理。最为关键的是，VFS 的可扩展性使得 Linux 系统能够轻松地支持多种多样的文件系统，以应对各种复杂的存储需求；同时，这种可扩展性也为后续 FUSE 在用户态实现调用内核态的功能提供了基础。

02 网络文件系统 NFS：首次突破内核态

随着计算需求的增长，单台计算机的性能逐渐无法满足日益增长的计算和存储要求。人们开始引入多台计算机，以分担负载并提高整体效率。

在这一场景下，一个应用程序往往需要访问分布在多台计算机上的数据。为了解决这一问题，人们提出了在网络中引入虚拟存储层的概念，将远程计算机的文件系统（如某个目录）通过网络接口挂载到本地计算机的节点上。这样做的目的是使本地计算机能够无缝地访问远程计算机的数据，就好像这些数据存储在本地一样。



具体来说，如果有一个需求是希望本地计算机能够访问远端的数据，那么可以将远端计算机的一个子目录通过网络接口虚拟出来，并挂载到本地计算机的一个节点上。在这个过程中，应用程序无需做任何修改，仍然通过标准的文件系统接口就可以访问这些路径，就像访问这些节点上的本地数据一样。

当应用程序对这些网络路径进行操作（如逐级目录查找）时，这些操作会被转化为网络请求，以 RPC（远程过程调用）的形式发送到远端计算机上执行。远端计算机在接收到这些请求后，会进行相应的操作（如查找文件、读取数据等），并将结果返回给本地计算机。

上述过程，就是一个简易的 NFS 协议实现，作为一种网络文件系统协议，NFS 为多台计算机之间的资源共享提供了高效的解决方案，用户可以像操作本地文件系统一样方便地挂载和访问远程文件系统。传统的文件系统通常是整个体系都运行在单机的内核态，而 NFS 首次打破了这一限制，服务端的实现结合了内核态与用户态。后续 FUSE 的设计，正是受到这一思路的启发。

03 FUSE：从内核到用户态的文件系统创新

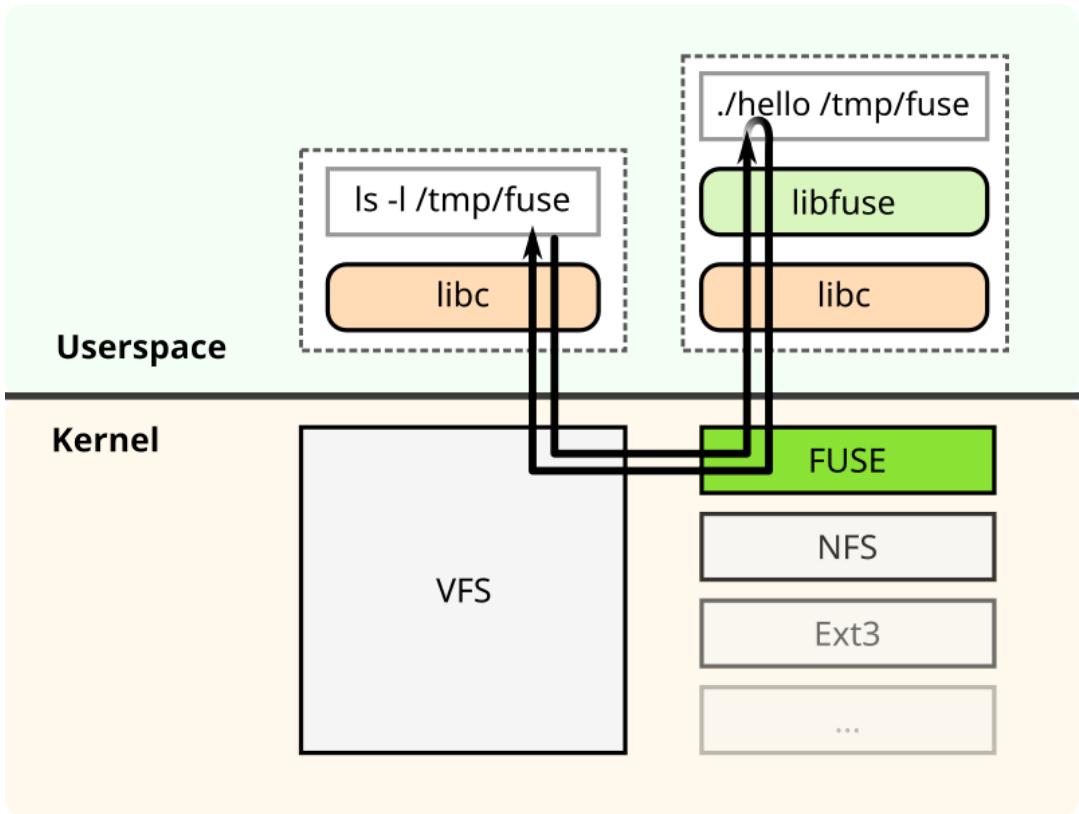
随着计算机技术的不断发展，许多新兴业务场景需要使用自定义文件系统。传统的内核态文件系统存在实现难度高和版本兼容性问题。NFS 的架构首次使文件系统突破了内核的限制。

基于此，有人提出了一个构想：是否可以将 NFS 网络协议移植到单机端，将服务端功能转移到用户态进程，同时保留客户端在内核中运行，用系统调用代替网络通讯，从而在用户态实现文件系统功能？这一想法最终催生了 FUSE（filesystem in userspace）的诞生。

在 2001 年，匈牙利的计算机学家 Miklos Szeredi 推出 FUSE，这是一种为开发者提供的框架，允许在用户空间实现文件系统。**FUSE 的核心分为两个部分：内核模块和用户态库（libfuse）。**

其内核模块作为操作系统内核的一部分，负责与 VFS 交互，将来自 VFS 的文件系统请求转发到用户态，并将用户态的处理结果返回给 VFS。这种设计使得 FUSE 能够在不修改内核代码的情况下，实现自定义文件系统的功能。

FUSE 的用户态库（libfuse）提供了与 FUSE 内核模块交互的 API 库，可以帮助用户实现一个运行在用户空间的守护进程（daemon）。守护进程负责处理来自内核的文件系统请求，并实现具体的文件系统逻辑。



在具体的实现中，用户态的守护进程（Daemon）与内核模块通过以下步骤协作完成文件操作：

请求接收 1. 内核模块会注册一个*字符设备（`/dev/fuse`）作为通信通道。守护进程通过调用 `read()` 函数主动从该设备读取请求： 2. 若内核的 FUSE 请求队列为空，`read()` 会进入阻塞状态，此时守护进程暂停执行并释放 CPU，直到队列中出现新请求（通过内核的等待队列机制实现）。 3. 当应用程序发起文件操作（如 `open`、`read`）时，内核模块将请求封装为特定格式的数据包，并插入请求队列，唤醒阻塞中的守护进程。

请求处理 守护进程从字符设备中读取到请求数据包后，根据操作类型（如读、写、创建文件）调用对应的用户态处理函数。

结果返回 处理完成后，守护进程将结果（如读取到的文件内容或错误码）按照 FUSE 协议格式序列化，并通过 `write()` 将数据包写回字符设备。

内核模块接收到响应后：

1. 解析数据包内容，将结果传递给等待中的应用程序；
2. 唤醒应用程序中被阻塞的系统调用，使其继续执行后续逻辑。

FUSE 的出现为文件系统的开发带来了革命性的变化，通过将文件系统的实现从内核态迁移到用户态，FUSE 大幅降低了开发的难度，提升了系统的灵活性和可扩展性，使其广泛地被应用于多种场景，如网络文件系统、加密文件系统以及虚拟文件系统等。

是的，FUSE 模块和 mount 操作是强相关的。

当你执行 `mount -t fuse myfs /mnt/xyz` 的时候，实际上就是在告诉 VFS：

“请把 `/mnt/xyz` 挂载到一个类型为 FUSE 的文件系统上，并且让它由我指定的用户态 daemon 来提供服务。”

也就是说：

FUSE 是一个“文件系统框架”（不是一个具体的文件系统）。

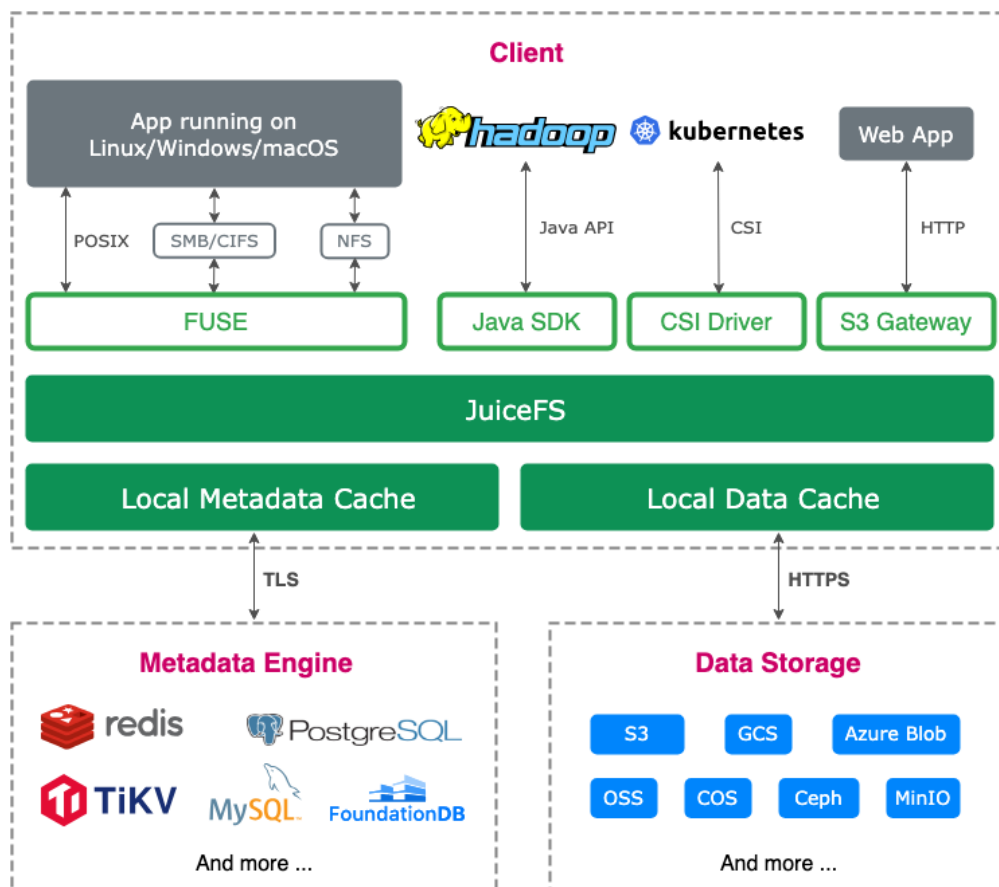
mount 命令是让 FUSE 注册并生效的入口点。

没有 mount，就没有 FUSE 文件系统的存在。

04 JuiceFS：FUSE 用户态分布式文件系统

2017 年，IT 基础设施全面进入云时代，架构面临前所未有的挑战，在此背景下，JuiceFS 诞生。作为一款基于对象存储的分布式文件系统，选择采用 FUSE 技术来构建其文件系统架构，以 FUSE 灵活的扩展性来应对云计算环境中的多样化需求。

通过 FUSE，JuiceFS 文件系统能够以 POSIX 兼容的方式挂载到服务器，将海量云端存储直接当做本地存储来使用，常见的文件系统指令（如 `ls`、`cp`、`mkdir` 等）都可以用来管理 JuiceFS 中的文件和目录。



以用户挂载 JuiceFS 后，open 其中一个文件的流程为例。请求首先通过内核 VFS，然后传递给内核的 FUSE 模块，经过 `/dev/fuse` 设备与 JuiceFS 的客户端进程通信。VFS 和 FUSE 的关系可以简单的看做客户端-服务器协议，VFS 作为客户端请求服务，用户态的 JuiceFS 扮演着服务器端的角色，处理这些请求。

具体步骤如下：

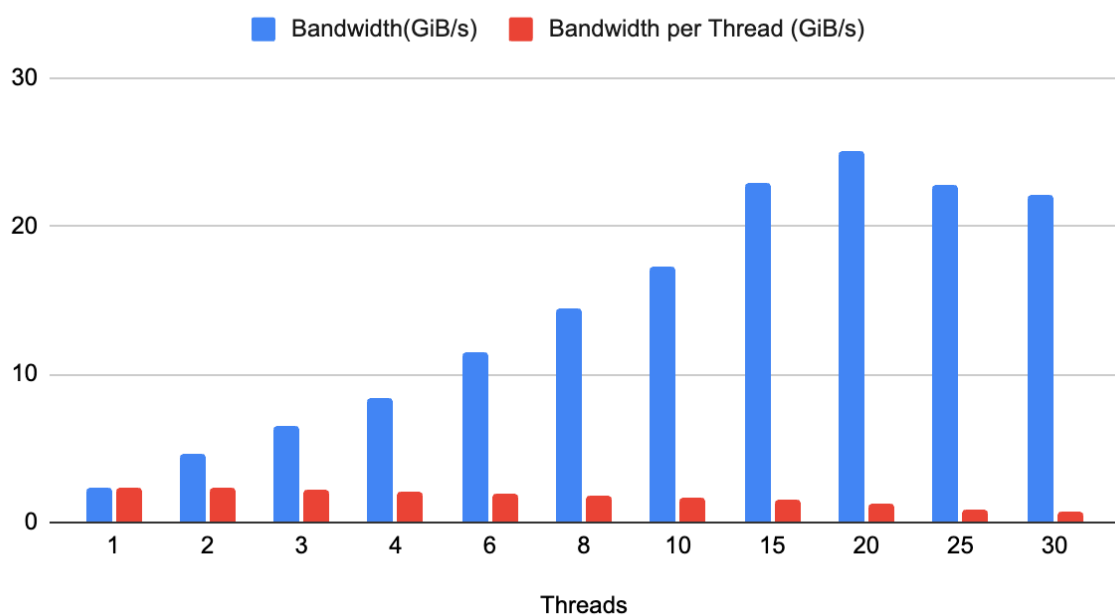
1. 当 JuiceFS mount 后, JuiceFS 内部的 go-fuse 模块会 open `/dev/fuse` 获取 mount fd, 并启动几个线程读取内核的 FUSE 请求;
2. 用户调用 open 函数, 通过 C 库和系统调用进入 VFS 层, VFS 层再将请求转给内核的 FUSE 模块;
3. 内核 FUSE 模块根据协议将 open 请求放入 JuiceFS mount 的 fd 对应的队列中, 并唤醒 go-fuse 的读请求线程, 等待处理结果;
4. 用户态的 go-fuse 模块读取 FUSE 请求并在解析请求后调用 JuiceFS 的对应实现;
5. go-fuse 将本次请求的处理结果写入 mount fd, 即放入 FUSE 结果队列, 然后唤醒业务等待线程;
6. 业务线程被唤醒, 得到本次请求的处理结果, 然后再返回到上层。

由于 FUSE 依赖于用户空间与内核之间的频繁切换, 许多人对其性能表现存有疑虑, 实际上并非完全如此, 我们使用 JuiceFS 进行了一组测试。

在一台1.5T内存、Intel Xeon 架构176核的机器上的 JuiceFS 文件系统中创建一个 512G 的空洞文件, 然后使用 fio 对其进行顺序读测试 ([挂载参数详情](#), fio 命令详情 (见文末)), 这样可以排除硬件磁盘的制约, 测试出 FUSE 文件系统相对极限的带宽。

- 可以看到单挂载点下的单线程带宽可以达到 2.4 GiB/s;
- 随着线程的增加, 带宽基本可以实现线性增长, 在 20 线程时达到了 25.1 GiB/s, 这个吞吐量已经可以满足绝大部分实际业务场景。

JuiceFS 顺序读吞吐量测试 (全内存)



在 FUSE 相关的使用上, JuiceFS 实现了平滑升级功能。通过保证 mount fd 的一致性, 让用户可以在不需要重新挂载文件系统、不中断业务的情况下, 升级 JuiceFS 版本或修改挂载参数, 详情请参考[平滑升级功能详解, 不停服即可更新](#)。

FUSE 也存在一些局限性, 比如进程访问 FUSE device 需要很高的权限, 尤其在容器环境中, 通常需要开启特权模式。另外, 容器通常是短暂且无状态的, 如果容器意外退出, 并且数据没有及时落盘, 会存在数据丢失的风险。因此针对 Kubernetes 场景, JuiceFS 提供了 [CSI 驱动](#) 使得业务可以以非特权容器访问 JuiceFS 文件系统, 并且 CSI 驱动管理 FUSE 进程的生命周期, 确保数据能够及时落盘不会出现丢失的问题。 [Kubernetes 数据持久化: 从零开始使用 JuiceFS CSI Driver](#)

05 小结

FUSE 通过将用户空间与内核空间解耦，为开发者提供了在用户空间实现文件系统的巨大灵活性和便利性。特别是在云计算和分布式存储等现代计算环境中，FUSE 使得构建和维护复杂的存储系统变得更加高效、可定制和易于扩展。

JuiceFS 正是基于 FUSE，在用户空间实现了高性能的分布式文件系统。未来，我们将继续深入探索 FUSE 的优化方法，不断提升文件系统的性能与可靠性，以应对日益复杂的存储需求，为用户提供更强大的数据管理能力。

希望这篇内容能够对你有一些帮助，如果有其他疑问欢迎加入 [JuiceFS 社区](#) 与大家共同交流。

-

FUSE 文件系统和 libfuse 介绍

前言

这几天为了解决浏览器产品中的一个问题，研究了一下 FUSE(Filesystem in Userspace)。通常，应用程序开发只需要使用系统 API 进行文件读写操作，不需要了解文件系统的细节。在 Chromium 中为了实现跨平台，甚至对各操作系统的文件 API 进行了封装。在代码中使用封装 API，都不需要了解各操作系统所提供的文件系统 API。但由于我们的浏览器产品中使用了 FUSE 进行加密存储，所以有必要了解 FUSE 和 libfuse。

FUSE 介绍

文件系统为应用程序提供了一个访问数据的通用接口。文件系统可以在用户态中实现，比如像鸿蒙这样的微内核。但大多数宏内核操作系统（如 Linux），文件系统是在内核态中实现，以保证性能。

然而，随着文件系统复杂性的增加，用户态文件系统的使用逐渐增多，特别是在快速开发和实验性研究领域。采用用户态文件系统有如下优势：

- 开发效率更高，易于维护和移植。
- 用户态中的错误影响范围有限，降低了系统崩溃的风险。
- 支持多平台的丰富编程语言和库。

而用户态文件系统最大的缺点是性能开销较大，特别是在用户态和内核态之间的通信和上下文切换时。这一点在后面介绍 FUSE 的架构时会有更详细的说明。

FUSE (Filesystem in Userspace) 是当前最广泛使用的**用户态文件系统**框架。据保守估计，至少有 100 种基于 FUSE 的文件系统可以在互联网上找到。

FUSE 得到广泛使用主要得益于其提供的简单 API。FUSE 项目由两个组件组成：由【**常规内核代码库维护的 fuse 内核模块**】和【**libfuse 用户空间库**】。libfuse 提供了与 FUSE 内核模块通信的参考实现。

对于应用开发者而言，通常只需要使用 libfuse，无需了解 fuse 内核模块。不过为了更好的使用 libfuse 开发文件系统，最好理解 FUSE 的高层设计，了解其实现的一些细节。

所以，本文先介绍 FUSE 的高层架构，并解释一些重要的实现细节，接着重点介绍 libfuse 的 API，并介绍在 UOS/Deepin 上的编译与运行示例。

FUSE 高层架构

FUSE 由内核部分和用户级守护进程组成。内核部分实现为一个 Linux 内核模块，当加载时，会向 Linux 的虚拟文件系统（VFS）注册一个 FUSE 文件系统驱动程序。该 FUSE 驱动程序充当由不同用户级守护进程实现的各种特定文件系统的代理。

除了注册一个新文件系统外，FUSE 的内核模块还注册了一个 `/dev/fuse` 块设备。该设备作为用户态 FUSE 守护进程与内核之间的接口。通常，守护进程从 `/dev/fuse` 读取 FUSE 请求，处理后将回复写回 `/dev/fuse`。

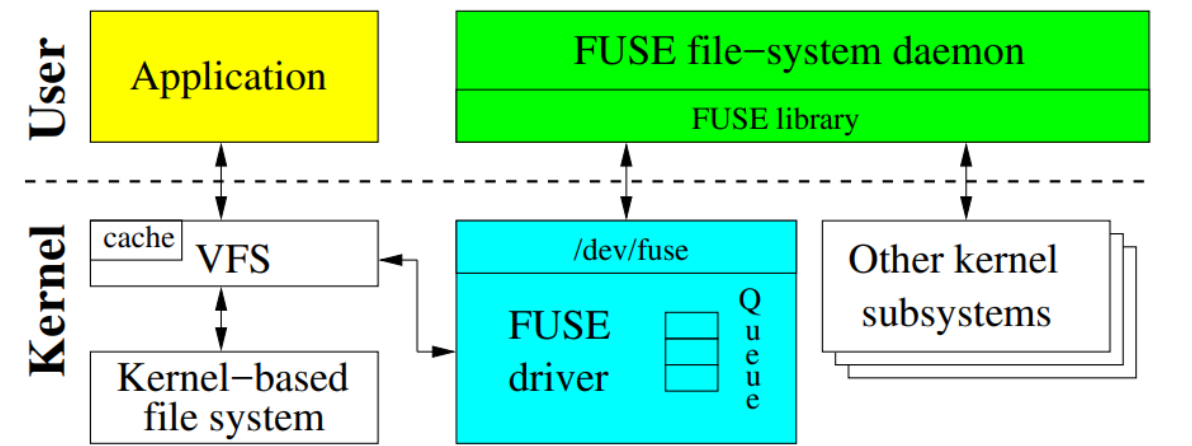


图1 FUSE 架构

FUSE 的高层架构如图 1 所示。当用户空间程序对挂载的 FUSE 文件系统发起操作时，其调用流程如下：

1. 用户程序通过系统调用请求文件系统操作（如读取文件）。
2. Linux 虚拟文件系统（VFS）将请求转发到 FUSE 的内核模块。
3. FUSE 内核模块将请求打包成 FUSE request 数据结构，存入内核的 FUSE 队列，同时将调用进程挂起等待结果。
4. 用户空间守护进程（**FUSE daemon**）从 `/dev/fuse` 设备读取请求，处理后再将结果写回 `/dev/fuse`。
5. FUSE 内核模块从 `/dev/fuse` 接收到结果后，将结果返回给挂起的进程。
6. 用户程序接收到文件系统操作的结果。

从上面的流程可以看出，每次文件操作都需要从用户态程序进入内核态，然后通过 FUSE 内核模块将请求发送到用户态的 FUSE 守护进程，处理完成后再切换回内核态，最后返回给用户程序。这个调用过程不仅是增加了调用层次，而且系统调用和上下文切换增加了额外的开销。请求被放入 FUSE 队列后，必须等待用户态守护进程来处理。守护进程可能由于调度延迟或处理能力不足而导致请求滞后。这些都会导致 FUSE 性能下降。

libfuse 介绍

libfuse 是一个用户空间库，作为用户空间程序与 Linux 内核中的 FUSE 模块之间的接口。libfuse 为开发者提供了操作文件系统所需的功能，使其能够通过一个简洁的 API 实现复杂的文件系统行为。

开发者使用 libfuse 来实现自定义的文件系统逻辑，而内核通过 fuse 模块与用户空间的 libfuse 进行通信。

libfuse 提供了一套 API，供用户空间文件系统与内核交互。这些 API 主要集中在对文件系统操作的实现上，如打开、读写文件、删除文件等。以下是 libfuse 提供的一些常用 API：

1. 文件系统操作的回调函数

开发者需要实现一些回调函数，这些回调函数处理用户空间的文件系统操作。每个文件系统操作（如读取、写入、创建文件等）都需要对应的回调函数。

`fuse_operations` 结构体：

该结构体定义了文件系统的各类操作（如 `read`, `write`, `create`, `unlink` 等）对应的回调函数。

示例：

```
struct fuse_operations oper = {
    .getattr    = my_getattr,
    .read       = my_read,
    .write      = my_write,
    // 其他操作...
};
```

每个操作函数都有对应的签名，例如：

- `getattr`：获取文件属性

```
int (*getattr)(const char *path, struct stat *stbuf);
```

- `read`：从文件中读取数据

```
int (*read)(const char *path, char *buf, size_t size, off_t offset, struct
fuse_file_info *fi);
```

2. 文件系统启动与关闭

- `fuse_main`：

启动 FUSE 文件系统并进入事件循环。开发者需要传入 `fuse_operations` 结构体和命令行参数来启动文件系统。